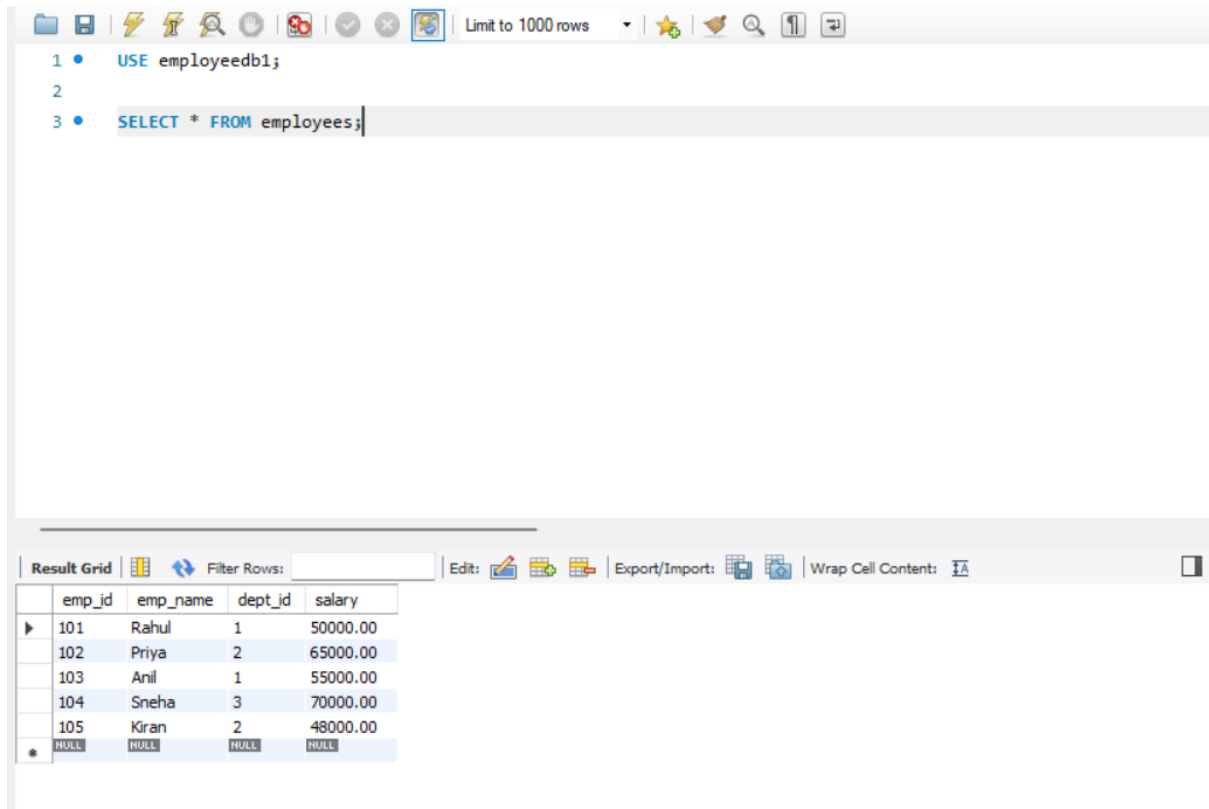


DBMS – MYSQL STORED PROCEDURES

T.PARINITA – 2520030562

Employee Database and Table Data



The screenshot displays the MySQL Workbench interface. The top toolbar includes icons for file operations, execution, and search. The SQL editor shows the following queries:

```
1 • USE employeeedb1;  
2  
3 • SELECT * FROM employees;
```

The bottom toolbar contains options for the Result Grid, Filter Rows, Edit, Export/Import, and Wrap Cell Content. The Result Grid shows the following data:

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00
* NULL	NULL	NULL	NULL

Procedure to Display All Employees

Limit to 1000 rows

```
1 • USE employeeedb1;
2
3 • SELECT * FROM employees;
4 • DROP PROCEDURE IF EXISTS GetEmployees;
5
6 DELIMITER $$
7
8 • CREATE PROCEDURE GetEmployees()
9 BEGIN
10     SELECT * FROM employees;
11 END $$
12
13 DELIMITER ;
14 • CALL GetEmployees();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [fA](#)

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

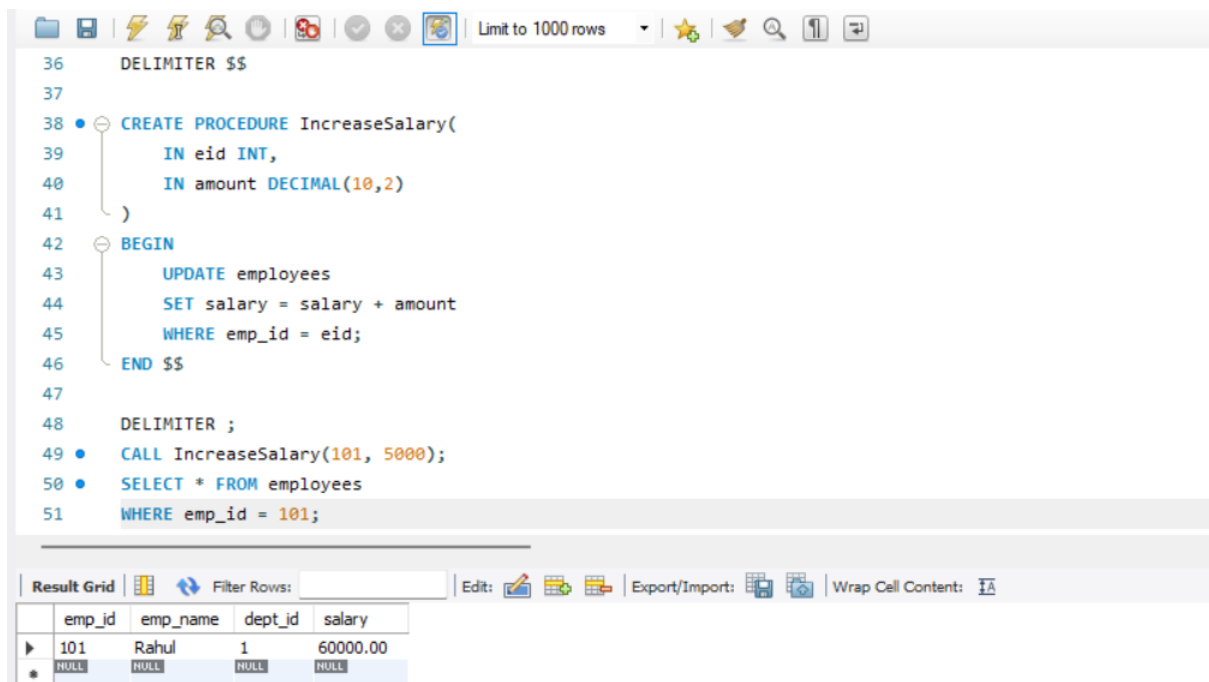
Procedure Using IN Parameter

```
16 • USE employeeedb1;
17
18 • DROP PROCEDURE IF EXISTS GetEmployeeById;
19
20 DELIMITER $$
21
22 • CREATE PROCEDURE GetEmployeeById(IN eid INT)
23 BEGIN
24     SELECT *
25     FROM employees
26     WHERE emp_id = eid;
27 END $$
28
29 DELIMITER ;
30 • CALL GetEmployeeById(102);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [fA](#)

emp_id	emp_name	dept_id	salary
102	Priya	2	65000.00

Procedure to Increase Employee Salary



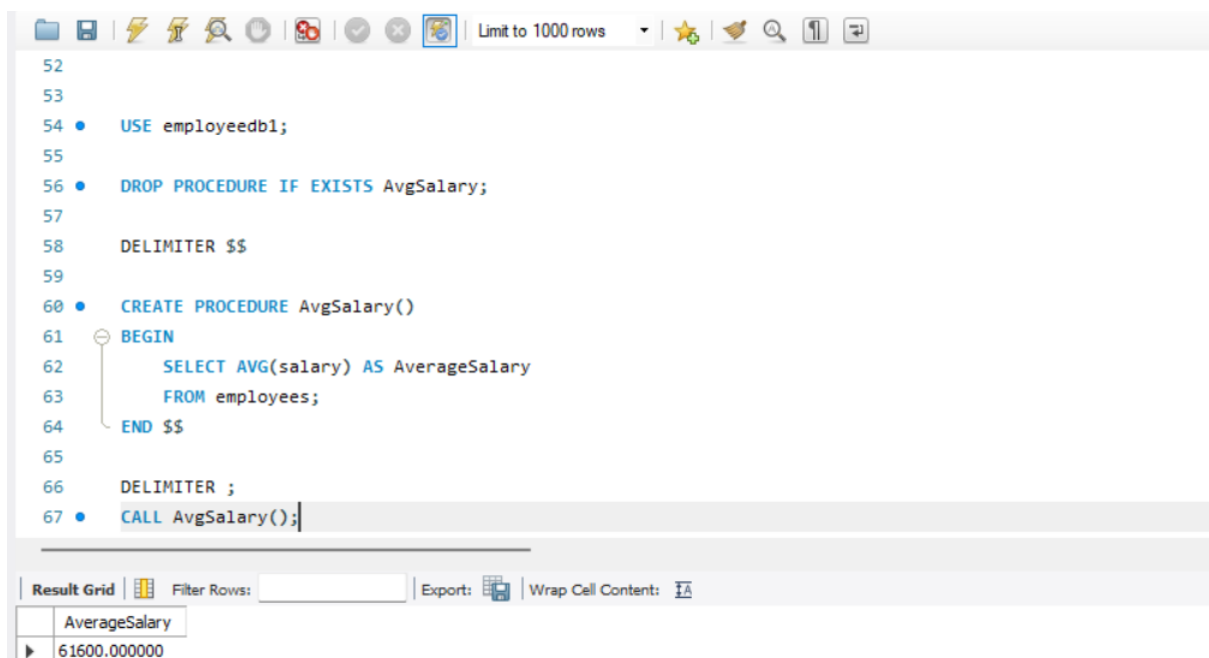
The screenshot shows a SQL IDE with a toolbar at the top. The main editor contains the following SQL code:

```
36 DELIMITER $$
37
38 CREATE PROCEDURE IncreaseSalary(
39     IN eid INT,
40     IN amount DECIMAL(10,2)
41 )
42 BEGIN
43     UPDATE employees
44     SET salary = salary + amount
45     WHERE emp_id = eid;
46 END $$
47
48 DELIMITER ;
49 CALL IncreaseSalary(101, 5000);
50 SELECT * FROM employees
51 WHERE emp_id = 101;
```

Below the editor is a "Result Grid" section with a "Filter Rows:" input and buttons for "Edit", "Export/Import", and "Wrap Cell Content". The result grid displays the following data:

emp_id	emp_name	dept_id	salary
101	Rahul	1	60000.00
NULL	NULL	NULL	NULL

Procedure Using Aggregate Function



The screenshot shows a SQL IDE with a toolbar at the top. The main editor contains the following SQL code:

```
52
53
54 USE employeeedb1;
55
56 DROP PROCEDURE IF EXISTS AvgSalary;
57
58 DELIMITER $$
59
60 CREATE PROCEDURE AvgSalary()
61 BEGIN
62     SELECT AVG(salary) AS AverageSalary
63     FROM employees;
64 END $$
65
66 DELIMITER ;
67 CALL AvgSalary();
```

Below the editor is a "Result Grid" section with a "Filter Rows:" input and buttons for "Export" and "Wrap Cell Content". The result grid displays the following data:

AverageSalary
61600.000000

Procedure Using JOIN

Limit to 1000 rows

```

97
98     DELIMITER $$
99
100 • CREATE PROCEDURE DepartmentAverageSalary()
101     BEGIN
102         SELECT d.dept_name,
103                COUNT(e.emp_id) AS EmployeeCount,
104                AVG(e.salary) AS AverageSalary
105         FROM employees e
106         JOIN departments d
107           ON e.dept_id = d.dept_id
108         GROUP BY d.dept_id, d.dept_name;
109     END $$
110
111     DELIMITER ;
112 • CALL DepartmentAverageSalary();

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	dept_name	EmployeeCount	AverageSalary
▶	CSE	2	77500.000000
	ECE	2	56500.000000
	EEE	1	70000.000000

IF...ELSE Salary Classification

```

121 • BEGIN
122     DECLARE sal DECIMAL(10,2);
123
124     SELECT salary INTO sal
125     FROM employees
126     WHERE emp_id = eid;
127
128     IF sal >= 60000 THEN
129         SELECT 'High Salary' AS Status;
130     ELSE
131         SELECT 'Low Salary' AS Status;
132     END IF;
133 END $$
134
135     DELIMITER ;
136 • CALL SalaryStatus(102);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	Status
▶	High Salary

OUT Parameter for Total Employee Count

```

151 • CALL EmployeeCount(@total);
152 • SELECT @total AS TotalEmployees;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	TotalEmployees			
▶	5			

INOUT Parameter

```

167 • CALL AddBonus(@bonus);
168
169 • SELECT @bonus AS UpdatedBonus;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	UpdatedBonus			
▶	15000.00			

Transaction

```

188 UPDATE employees
189 SET salary = salary + amount
190 WHERE emp_id = toEmp;
191
192 COMMIT;
193 END $$
194
195 DELIMITER ;
196 • SELECT *
197 FROM employees
198 WHERE emp_id IN (101,102);
199 • CALL TransferSalaryAmount(101,102,2000);
200
201 • SELECT *
202 FROM employees
203 WHERE emp_id IN (101,102);

```

Result Grid		Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
emp_id	emp_name	dept_id	salary		
▶ 101	Rahul	1	151000.00		
102	Priya	2	69000.00		
*	NULL	NULL	NULL		

STUDENT PRACTICE QUESTIONS

Employees Above Given Salary

```
207 • DROP PROCEDURE IF EXISTS EmployeesAboveSalary;
208
209 DELIMITER $$
210
211 • CREATE PROCEDURE EmployeesAboveSalary(IN min_salary DECIMAL(10,2))
212 BEGIN
213     SELECT *
214     FROM employees
215     WHERE salary > min_salary;
216 END $$
217
218 DELIMITER ;
219 • CALL EmployeesAboveSalary(50000);
220
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

emp_id	emp_name	dept_id	salary
101	Rahul	1	157000.00
102	Priya	2	73000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00

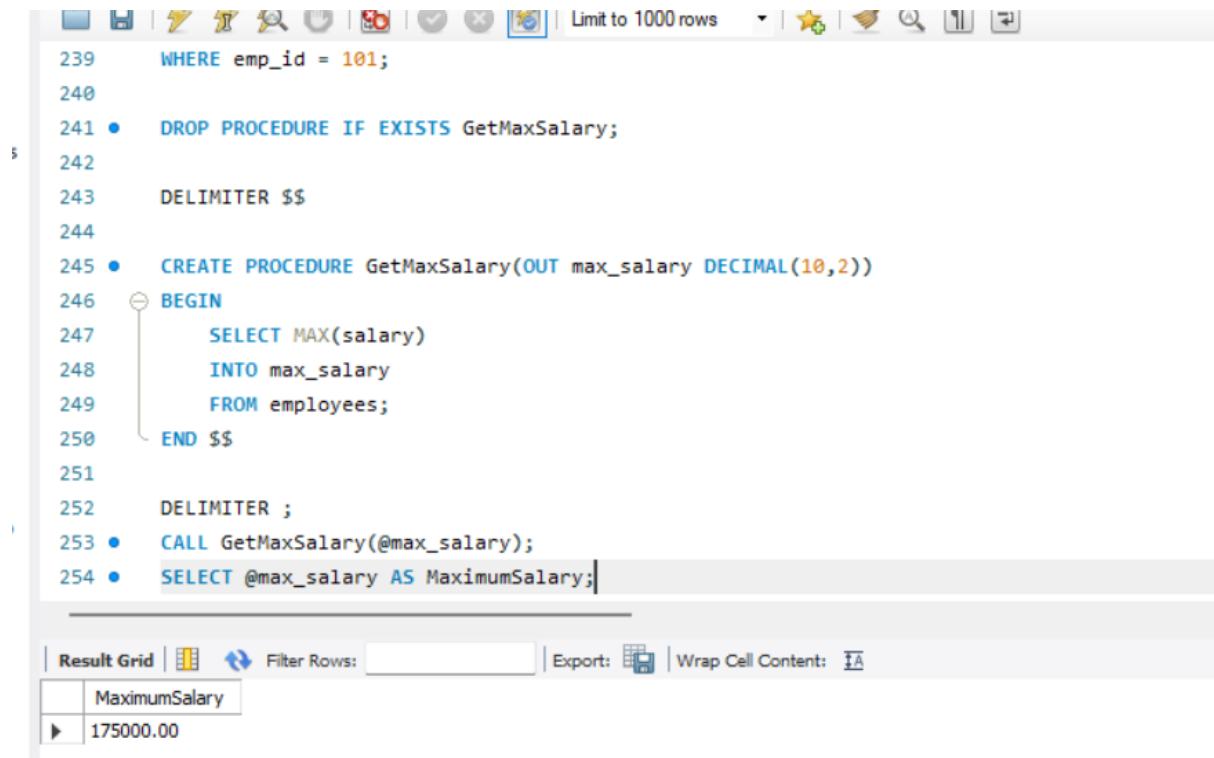
Update Employee Department

```
224
225 • CREATE PROCEDURE UpdateEmployeeDepartment(
226     IN employee_id INT,
227     IN new_department_id INT
228 )
229 BEGIN
230     UPDATE employees
231     SET dept_id = new_department_id
232     WHERE emp_id = employee_id;
233 END $$
234
235 DELIMITER ;
236 • CALL UpdateEmployeeDepartment(101, 2);
237 • SELECT *
238 FROM employees
239 WHERE emp_id = 101;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

emp_id	emp_name	dept_id	salary
101	Rahul	2	166000.00
*	NULL	NULL	NULL

Maximum Salary Using OUT

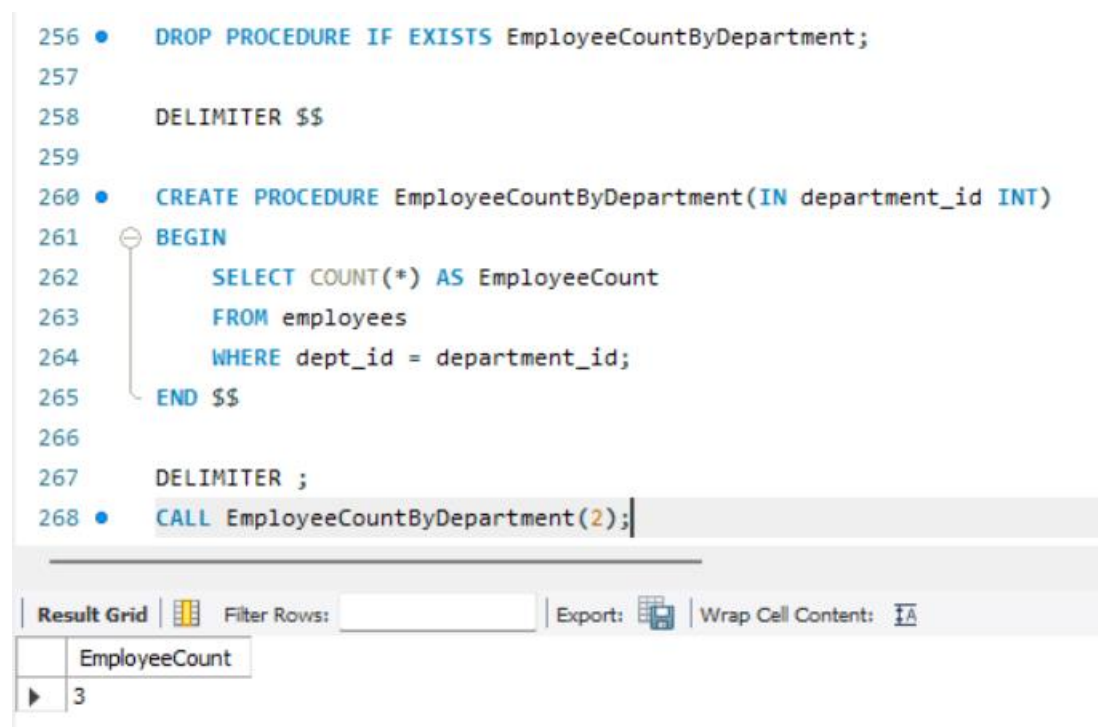


```
239 WHERE emp_id = 101;
240
241 • DROP PROCEDURE IF EXISTS GetMaxSalary;
242
243 DELIMITER $$
244
245 • CREATE PROCEDURE GetMaxSalary(OUT max_salary DECIMAL(10,2))
246 BEGIN
247     SELECT MAX(salary)
248     INTO max_salary
249     FROM employees;
250 END $$
251
252 DELIMITER ;
253 • CALL GetMaxSalary(@max_salary);
254 • SELECT @max_salary AS MaximumSalary;
```

Result Grid

MaximumSalary
175000.00

Employee Count of a Department



```
256 • DROP PROCEDURE IF EXISTS EmployeeCountByDepartment;
257
258 DELIMITER $$
259
260 • CREATE PROCEDURE EmployeeCountByDepartment(IN department_id INT)
261 BEGIN
262     SELECT COUNT(*) AS EmployeeCount
263     FROM employees
264     WHERE dept_id = department_id;
265 END $$
266
267 DELIMITER ;
268 • CALL EmployeeCountByDepartment(2);
```

Result Grid

EmployeeCount
3

JOIN Employee + Department Details


```

275 • CREATE PROCEDURE EmployeeDepartmentDetails()
276 BEGIN
277     SELECT
278         e.emp_id,
279         e.emp_name,
280         e.salary,
281         d.dept_id,
282         d.dept_name
283     FROM employees e
284     JOIN departments d
285     ON e.dept_id = d.dept_id;
286 END $$
287
288 DELIMITER ;
289 • CALL EmployeeDepartmentDetails();

```

emp_id	emp_name	salary	dept_id	dept_name
101	Rahul	187000.00	2	ECE
102	Priya	93000.00	2	ECE
103	Anil	55000.00	1	CSE
104	Sneha	70000.00	3	EEE
105	Kiran	48000.00	2	ECE

Departments With Average Salary > 55000

```

306 • CALL DepartmentsHighAverageSalary();

```

dept_id	AverageSalary
2	112666.666667
3	70000.000000

IF...ELSE Salary Classification

```

332 • CALL ClassifySalary(101);

```

Salary	SalaryLevel
199000.00	High

Average Salary of Department

```

333
334 • DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
335
336 DELIMITER $$
337
338 • CREATE PROCEDURE DepartmentAverageSalary(IN department_id INT)
339 BEGIN
340     SELECT AVG(salary) AS AverageSalary
341     FROM employees
342     WHERE dept_id = department_id;
343 END $$
344
345 DELIMITER ;
346 • CALL DepartmentAverageSalary(2);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

AverageSalary
119333.333333

INOUT Bonus

```

360 • SET @salary = 50000;
361 • CALL ApplyBonus(@salary);
362 • SELECT @salary AS SalaryAfterBonus;
363

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

SalaryAfterBonus
55000.00

Transaction With Two Updates

```

392 • CALL TransferSalary(101, 102, 2000);
393 • SELECT *
394 FROM employees
395 WHERE emp_id IN (101, 102);

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

emp_id	emp_name	dept_id	salary
101	Rahul	2	225000.00
102	Priya	2	125000.00
NULL	NULL	NULL	NULL